

Part 1

```
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $ cd deliverables
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/deliverables (main) $ ls ../deliverables/
README.md sbom_syft_spdx.json sbom_trivy_cdx.json vuln_analysis_grype.txt
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/deliverables (main) $
```

How many components each tool reported (Syft vs. Trivy)

Syft: 130 components

```
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ jq '.packages | length' ../deliverables/sbom_syft_spdx.json
130
```

Trivy: 104 components

```
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ jq '.components | length' ../deliverables/sbom_trivy_cdx.json
104
```

One difference you notice between the SPDX SBOM and the CycloneDX SBOM (format, fields, component count, etc.), and

When answering the previous question about the number of components that each tool reported, I noticed that the spdx SBOM uses the term “packages” for the list of elements, while CycloneDX SBOM uses "components." Additionally, the total number of elements identified differs (130 vs. 104), likely due to differences in how each tool detects and categorizes dependencies.

Part 2

```
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $ cd deliverables
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/deliverables (main) $ ls ../deliverables/
README.md sbom_syft_spdx.json sbom_trivy_cdx.json vuln_analysis_grype.txt
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ grype sbom:../deliverables/sbom_syft_spdx.json -o table > ../deliverables/vuln_analysis_grype.txt
✓ Vulnerability DB [updated]
✓ Scanned for vulnerabilities [5 vulnerability matches]
  └─ by severity: 0 critical, 1 high, 3 medium, 1 low, 0 negligible
     by status: 5 fixed, 0 not-fixed, 0 ignored
@japievkaur → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $
```

In your report, include a table for the top 5 vulnerabilities that includes the following:

CVE	Severity	Component	Version	Comment
CVE-2026-41066	High	lxml	5.2.2	Issue with XML parsing that could let malicious input cause problems
CVE-2026-39892	Medium	cryptography	46.0.5	Weakness in encryption handling that could impact data security

CVE-2025-71176	Medium	pytest	8.3.3	Issue in the testing tool that might allow unintended code execution
CVE-2026-34073	Low	cryptography	46.0.5	Issue in how encryption is handled with limited impact
CVE-2026-25645	Medium	requests	2.32.4	Issue with handling web requests that could expose the app to risk

```
head -20 ../deliverables/vuln_analysis_grype.txt
```

Result of the command:

NAME	INSTALLED	FIXED IN	TYPE	VULNERABILITY	SEVERITY	EPSS	RISK
lxml	5.2.2	6.1.0	python	GHSA-vfmq-68hx-4jfw	High	< 0.1% (9th)	< 0.1
cryptography	46.0.5	46.0.7	python	GHSA-p423-j2cm-9vmq	Medium	< 0.1% (5th)	< 0.1
pytest	8.3.3	9.0.3	python	GHSA-6w46-j5rx-g56g	Medium	< 0.1% (0th)	< 0.1
cryptography	46.0.5	46.0.6	python	GHSA-m959-cc7f-wv43	Low	< 0.1% (0th)	< 0.1
requests	2.32.4	2.33.0	python	GHSA-gc5v-m9x4-r6x2	Medium	< 0.1% (0th)	< 0.1

CVE-2026-41066 is caused by how the cryptography library handles certain types of input data, which can lead to a buffer overflow and potentially crash the program or create security issues.

Reflection

This lab showed how SBOMs help make all parts of a system more visible. I got experience working with both Syft and Trivy to generate SBOMs, and saw that different tools can find different components, which is why using multiple tools helps with defense in depth. The Grype scan made it clear that common libraries can have vulnerabilities, so it's important to continuously check them.

From my own experience, during my internship at State Farm I used Snyk for vulnerability scanning. If there were high-severity issues, they would block our pipeline until we fixed them, which reinforced how important it is to address vulnerabilities early.